ONSURITY — ENGINEERING — BENGALURU

Software Development Engineer II, Java

The level where you stop being handed the design and start writing it. You'll own features end to end — build them, ship them, and carry them in production. **We expect you to be the reason something works, not just the person who coded it.**

LEVEL	TRACK	EXPERIENCE	LOCATION	STACK
SDE II	Individual contributor	3–5 years	Bengaluru	Java · Node · React

ABOUT US

Healthcare benefits shouldn't be a luxury

Onsurity is a fast-growing employee healthcare benefits platform. We give SMEs, start-ups and enterprises flexible healthcare subscriptions — discounted medicine, health checkups, fitness plans, free doctor teleconsultations and insurance, in one membership. Our plans aren't limited to full-time employees; we also cover contractual workers, interns, freelancers and consultants, because inclusion is the whole point.

We've raised a **$45M Series B led by Creaegis**, count cricketing legend **Anil Kumble** as a strategic advisor and investor, featured in season one of Disney+ Hotstar's *The Great Indian Disruptors*, and were named a **Dun & Bradstreet Start-up 50 Trailblazer**. More at onsurity.com — or find us on LinkedIn, Instagram and YouTube.

THE ROLE

What this job actually is

We're hiring engineers who can take a half-specified feature, close the gaps themselves, and ship something that holds up.

You'll work inside a domain — claims, wallet, subscriptions, provider network or payments — and own real parts of it. Someone more senior owns the domain; the features, services and endpoints inside it are genuinely yours, including after they're live.

You'll write the low-level design yourself and take it to review: schemas, API contracts, state transitions, the error cases. You'll sit in the high-level design discussions your seniors are running and be expected to push on them, not just take notes.

Java and Spring Boot are the centre of gravity, but the boundary of your work is the feature, not the language. When a feature needs a React screen or a Node service, the expectation is that you build it rather than wait for someone else to free up.

The problems arrive incomplete. Insurance partners, hospitals, regulators and finance all constrain how a feature behaves and they don't agree with each other. At this level you're not expected to resolve all of that alone — you are expected to notice it, ask the right person, and not build on an assumption you never checked.

THE LEVEL

What SDE II means here

The step up from SDE I is ownership and judgement, not output. These six are what the level asks you to carry.

SCOPE	A feature or a service, end to end. You write the LLD and defend it in review.
AMBIGUITY	The problem arrives half-specified. You close the gaps yourself and flag the ones you can't.
INFLUENCE	Your team's codebase, and the engineers reviewing alongside you.
JUDGEMENT	You make the call on your own work, and know which decisions belong above your line.
CONSEQUENCE	You're on call for what you built, and you fix the cause rather than the symptom.
MULTIPLIER	You review well, and the engineers newer than you get better for having you there.

YOUR WORK

What you'll own

DESIGN

- Own the **LLD** for your features — schemas, class and API contracts, state transitions, and the error cases that only show up under load.
- Contribute properly to the **HLD** your seniors are writing. Ask the awkward question in the room, and own a piece of the result.
- Write the trade-off down. "We chose X over Y because Z" saves the next engineer a week, and saves you the argument twice.
- Take the design through review before you build it. Rework after the fact is the expensive kind.

BUILD

- Build and maintain Java services other teams depend on, with tests that would actually catch a regression.
- Ship the React screen or the Node endpoint when the feature needs it, rather than stopping at the API boundary.
- Handle money and health data carefully — correct, traceable, and safe to retry when something fails halfway through.
- Change a schema without taking the feature down, and know how you'd roll it back before you start.

RUN

- Be on call for what you built. You should know something is wrong before support forwards you a ticket.

- Add the metrics, logs, traces and alerts as part of the feature, not as a follow-up nobody schedules.
- Debug production with incomplete information — narrow it down yourself before escalating, and bring what you've ruled out.
- Notice when something got slower or more expensive, and find out why rather than filing it as normal.

MULTIPLY

- Review code usefully — aimed at what will hurt in six months, not at formatting.
- Mentor the SDE Is around you. Answer the question, then show them how you'd have found the answer.
- Raise problems early. Surprises at the end of a sprint are the ones that cost other people their week.
- Say no with a reason, including to your own earlier decisions when the evidence changes.

THE BAR

NON-NEGOTIABLE

How we expect you to operate

Skills get you the interview. These get you the offer, and they're the things we won't compromise on.

High ownership

You drive what you build from design to production to the boring long tail afterwards. You instrument it, you watch it, you get paged for it. No handing it over at the merge commit.

Highly motivated and self-driven

A bias toward action and a real sense of accountability. You find the next most useful thing without being assigned it, and you say clearly when something slips.

Extensive AI use

You already use Copilot, Claude or ChatGPT heavily — development, code review, debugging, documentation — and you still read every line you ship, because you're the one accountable for it.

Fast, with a floor under it

You ship in days, not quarters, and you don't buy that speed by skipping tests or leaving the mess for the next person.

Direct and low-ego

You disagree in the design review, not in the retro. You change your mind when someone is right, and you write things down so the team doesn't have to keep asking you.

AI

How we expect you to use AI

AI-assisted development is the default here, not a novelty. We expect it to show up in what you get done.

- Use Copilot, Claude or ChatGPT extensively and daily – scaffolding, tests, refactors, code review, debugging, documentation, migrations.
- Read and own every line you ship. "The AI wrote it" is not an explanation for a production incident.
- Know where it's weak – unfamiliar domain rules, security-sensitive paths, and anything touching money or member health data. Verify those yourself.
- Start building the workflow, not just the habit: the prompts, context and repo conventions that make your team faster, not only you.
- Have opinions about AI in the product too, not only in the toolchain. We're building it into what members use.

MINIMUM

Minimum qualifications

- **3–5 years** of professional software development, the majority of it in Java.
- **Strong Java** (8 or higher; 17+ preferred) and real Spring / Spring Boot experience, not tutorial-level familiarity.
- **LLD you can hand over** – schemas, contracts, state transitions and edge cases another engineer could build from.
- **HLD you can hold your end of** – service boundaries, data flow, scale and failure modes, argued out loud in a review rather than nodded through.
- **Productive in a codebase you didn't write** – a real REST or microservice system, not only greenfield work.
- **SQL you can optimise** – you can find a slow query and fix it yourself. Some MongoDB exposure.
- **Kafka** or a similar event-driven system, run in production rather than prototyped – consumers you wrote, deployed and have debugged when they fell behind.
- **Kubernetes and containers** – you deploy, debug and scale your own service.
- **Monitoring and observability** – the metrics, logs, traces, dashboards and alerts on your services were added by you (Prometheus, Grafana, Sentry, ELK or equivalent), not inherited from someone else.
- **Some Node.js or React** – enough to carry a feature across the boundary instead of handing it off.
- **Extensive daily AI use** already built into how you work.
- Experience **supporting what you shipped** in production, including being on call for it.
- **Bachelor's degree** in Computer Science, Engineering or a related field, or equivalent practical experience.
- **Clear written and spoken communication** in a cross-functional team.

PREFERRED

Preferred qualifications

- You've led a feature that took more than one engineer to deliver.
- You've measurably improved an existing system – reliability, latency or cost – and can show the before and after.
- You've been in a production incident and can explain what you actually did during it.

- You've mentored an intern or a junior engineer.
- Production Node.js services, TypeScript end to end, or React Native.
- You've built with LLM APIs, not only used AI tooling.
- Payments or fintech background – gateways, refunds, reconciliation.
- Health insurance domain exposure – cashless approvals, claims, renewals.
- Experience handling regulated or sensitive customer data.

OUR STACK

What you'd work with

You'll be expected to be effective across most of this, and to pick up the rest without a long ramp.

language	Java 17+, Node.js, TypeScript
backend	Spring Boot, REST APIs, JUnit
frontend	React web, React Native mobile app
data	MySQL, PostgreSQL, MongoDB, Redis
streaming	Kafka
platform	AWS, Docker, Kubernetes, GitHub Actions
observability	Prometheus, Grafana, Sentry, ELK
ai	Copilot, Claude, ChatGPT — in the daily workflow

THE BAR, TESTED

What we'll assess

Six dimensions. We index on judgement and evidence, not on how many technologies you can name.

Coding

Correct, readable code under time pressure — and the restraint to keep the solution as simple as the problem allows.

Low-level design

The round that matters most at this level. Schemas, contracts, state transitions, concurrency, and whether a new requirement mid-round costs you a rewrite.

High-level design

A real Onsurity-shaped problem. Sensible boundaries, data flow and failure cases — we're reading participation and instinct, not a finished architecture.

Debugging

A production symptom with incomplete information. What you check first, how you narrow it, and how you stop it recurring.

Ownership deep-dive

Something you owned. Why those decisions, what broke, what it cost, and what you'd do differently now.

Bar and collaboration

How you handle disagreement and being wrong, and whether the engineers around you get better for having you there.

READ TWICE

This role is probably not for you if

- You want the design handed to you. At this level you write the LLD, and we'll ask you to defend it.
- You'd rather hand a service off once it's live. Ownership here includes the boring long tail and the 2 a.m. page.
- You want to stay strictly backend and never touch a screen or a pipeline. Features here don't respect that line.
- You need a long ramp before you contribute. We'd expect you shipping meaningfully inside your first few weeks.
- You're uncomfortable working where a bug can delay someone's hospital admission. That's the seriousness this team runs at.

EQUAL OPPORTUNITY

Onsurity is an equal-opportunity employer. We hire on capability and evidence, and we're committed to an inclusive workplace — the same philosophy behind healthcare plans that cover the people other plans leave out.